Федеральное государственное автономное образовательное учреждение высшего образования

«Национальный исследовательский технологический университет «МИСИС»

Институт компьютерных наук (ИКН)

Кафедра Инфокоммуникационных технологий (ИКТ)

# Лабораторная работа №2

по дисциплине «Технологии программирования»

на тему «Основы ООП»

Выполнил:
студент группы БИВТ-25-4-5

**Фунтусов Дмитрий Максимович**

Проверил:
ассистент

**Гласов Александр Васильевич**

Москва, 2026

**Цель работы:** изучить и применить на практике основные принципы объектно-ориентированного программирования — абстракцию, инкапсуляцию, наследование и полиморфизм — при проектировании иерархии классов для компьютерной игры (пошаговой стратегии). Освоить разницу между абстрактным классом и интерфейсом, научиться строить корректную иерархию наследования и реализовывать интерфейсы на языке Python.

## Ход выполнения работы

### 1) Создание проекта

Для работы был выбран язык Python и среда PyCharm. Создан проект `labOOPPython`, в котором все классы размещены в одном файле `lab2.py`. В Python нет отдельного ключевого слова `interface`, поэтому интерфейсы реализуются через абстрактные классы модуля `abc` (класс `ABC` и декоратор `@abstractmethod`).

### 2) Интерфейсы Attacker и Moveable

Созданы два интерфейса:

- `Attacker` с методом `attack(unit)` — контракт «умею атаковать юнита».
- `Moveable` с методом `move_to(x, y)` — контракт «умею перемещаться в точку».

Интерфейсы описывают роль, а не сущность. `Archer` и `Fort` не связаны общим родителем (один наследуется от `Unit`, другой от `Building`), но оба умеют атаковать, поэтому эта роль вынесена в интерфейс. Методы помечены `@abstractmethod`, поэтому класс, который не реализует их, нельзя создать.

```
from abc import ABC, abstractmethod



class Attacker(ABC):
    @abstractmethod
    def attack(self, unit: "Unit") -> None:
        pass



class Moveable(ABC):
    @abstractmethod
    def move_to(self, x: int, y: int) -> None:
        pass
```

Листинг 1 – Интерфейсы Attacker и Moveable

### 3) Класс GameObject

Создан абстрактный базовый класс `GameObject`, от которого наследуются все игровые объекты:

- Поля `_id`, `_name`, `_x`, `_y` начинаются с подчёркивания: так в Python принято обозначать защищённые поля, к которым не следует обращаться снаружи класса.
- Методы `get_id`, `get_name`, `get_x`, `get_y` — геттеры из задания.
- Класс наследуется от `ABC`, то есть задуман как абстрактный: «просто игрового объекта» не бывает, есть только конкретные юниты и постройки.

```
class GameObject(ABC):
    def __init__(self, id: int, name: str, x: int, y: int):
        self._id = id
        self._name = name
        self._x = x
        self._y = y

    def get_id(self) -> int:
        return self._id

    def get_name(self) -> str:
        return self._name

    def get_x(self) -> int:
        return self._x

    def get_y(self) -> int:
        return self._y
```

Листинг 2 – Класс GameObject

### 4) Класс Unit

Класс `Unit` наследуется от `GameObject` и описывает юнитов:

- Конструктор вызывает `super().__init__(...)` для инициализации родителя и сохраняет здоровье в поле `_hp`.
- `is_alive()` — возвращает `True`, если здоровье больше нуля.
- `get_hp()` — возвращает текущее здоровье.
- `receive_damage(damage)` — уменьшает здоровье на величину урона. Функция `max(0.0, ...)` не позволяет здоровью уйти ниже нуля.

```
class Unit(GameObject, ABC):
    def __init__(self, id: int, name: str, x: int, y: int, hp: float):
        super().__init__(id, name, x, y)
        self._hp = hp

    def is_alive(self) -> bool:
        return self._hp > 0

    def get_hp(self) -> float:
        return self._hp

    def receive_damage(self, damage: float) -> None:
        self._hp = max(0.0, self._hp - damage)
```

Листинг 3 – Класс Unit

### 5) Класс Archer

Класс `Archer` наследует `Unit` и реализует оба интерфейса — `Attacker` и `Moveable`. Это возможно, потому что Python допускает множественное наследование:

- Конструктор принимает здоровье (по умолчанию 100) и урон (по умолчанию 15) и передаёт общие параметры родителю.
- `attack(unit)` — проверяет, что сам лучник жив и что цель жива; если всё в порядке, наносит урон через `unit.receive_damage(...)`. Иначе выводит сообщение.
- `move_to(x, y)` — если лучник жив, меняет координаты и выводит новую позицию.

`Archer` — одновременно юнит (наследует `Unit`) и умеет атаковать и двигаться (реализует интерфейсы).

```
class Archer(Unit, Attacker, Moveable):
    def __init__(self, id: int, name: str, x: int, y: int,
                 hp: float = 100.0, damage: float = 15.0):
        super().__init__(id, name, x, y, hp)
        self._damage = damage


    def attack(self, unit: Unit) -> None:
        if not self.is_alive():
            print(f"{self.get_name()} мёртв и не может атаковать")
        elif not unit.is_alive():
            print(f"цель {unit.get_name()} уже мертва")
        else:
            unit.receive_damage(self._damage)


    def move_to(self, x: int, y: int) -> None:
        if self.is_alive():
            self._x, self._y = x, y
            print(f"{self.get_name()} at {(self._x, self._y)}")
        else:
            print(f"{self.get_name()} мертв и не может двигаться")
```

Листинг 4 – Класс Archer

## 6) Класс Building

Класс `Building` наследуется от `GameObject` и описывает постройку:

- Поле `_built` хранит, построено ли здание. Параметр `built` в конструкторе по умолчанию равен `False`.
- `is_built()` — возвращает, построено ли здание.
- `build()` — завершает строительство: переводит здание в состояние «построено».

```
class Building(GameObject, ABC):
    def __init__(self, id: int, name: str, x: int, y: int, built: bool =
False):
        super().__init__(id, name, x, y)
        self._built = built


    def is_built(self) -> bool:
        return self._built


    def build(self) -> None:
        self._built = True
```

Листинг 5 – Класс Building

## 7) Класс Fort

Класс `Fort` наследует `Building` и реализует `Attacker`:

- Конструктор вызывает конструктор родителя и сохраняет урон (по умолчанию 30).
- `attack(unit)` — если крепость не построена, стрелять нельзя, и выводится сообщение; если цель жива, ей наносится урон; если цель уже мертва, выводится сообщение.

Ключевая деталь: недостроенная крепость не стреляет, потому что атака защищена проверкой `if not self.is_built()`.

```
class Fort(Building, Attacker):
    def __init__(self, id: int, name: str, x: int, y: int,
                 built: bool = False, damage: float = 30.0):
        super().__init__(id, name, x, y, built)
        self._damage = damage


    def attack(self, unit: Unit) -> None:
        if not self.is_built():
            print(f"стрелять нельзя, крепость {self.get_name()} не
построена")
        elif unit.is_alive():
            unit.receive_damage(self._damage)
        else:
            print(f"{self.get_name()}: цель уже мертва")
```

Листинг 6 – Класс Fort

### 8) Класс MobileHouse

Класс `MobileHouse` наследует `Building` и реализует `Moveable`:

- Конструктор передаёт параметры в `Building`.
- `move_to(x, y)` — если дом построен (`is_built()`), меняет координаты и выводит новую позицию. Если не построен — выводит сообщение, что перемещение невозможно.

И `Archer`, и `MobileHouse` реализуют `Moveable`, но с разными проверками: лучник — по `is_alive()`, дом — по `is_built()`.

```
class MobileHouse(Building, Moveable):
    def __init__(self, id: int, name: str, x: int, y: int, built: bool =
False):
        super().__init__(id, name, x, y, built)


    def move_to(self, x: int, y: int) -> None:
        if not self.is_built():
            print(f"дом {self.get_name()} не построен, его перемещение
невозможно")
            return
        self._x, self._y = x, y
        print(f"{self.get_name()} at {(self._x, self._y)}")
```

Листинг 7 – Класс MobileHouse

**9) Демонстрационный блок**

В блоке `if __name__ == "__main__"` созданы пять объектов: лучник Robin, два вражеских лучника (Enemy с 20 здоровья и Enemy1 с 50), крепость Fort1 и дом house. Показано:

- Лучник трижды атакует Enemy (урон 15): здоровье падает с 20 до 5, затем до 0, а третья атака сообщает, что цель уже мертва.
- Лучник перемещается в точку (6, 7).
- Не построенные крепость и дом отказываются стрелять и двигаться, пока не вызван метод `build()`.
- После постройки крепость наносит два выстрела по 30: здоровье Enemy1 падает с 50 до 20, затем до 0.
- Построенный дом перемещается в точку (8, 7).

```
if __name__ == "__main__":
    archer = Archer(1, "Robin", 0, 0)
    enemy = Archer(2, "Enemy", 5, 5, hp=20)
    enemy1 = Archer(5, "Enemy1", 5, 5, hp=50)
    fort = Fort(3, "Fort1", 10, 10)
    house = MobileHouse(4, "house", 1, 1)


    archer.attack(enemy)
    archer.move_to(6, 7)
    archer.attack(enemy)
    archer.attack(enemy)


    fort.attack(enemy1)       # крепость не построена
    house.move_to(7, 7)       # дом не построен


    fort.build()
    house.build()


    fort.attack(enemy1)
    fort.attack(enemy1)
    house.move_to(8, 7)
```

Листинг 8 – Демонстрационный блок (сокращённо)

## 10) Работа программы

Результат запуска программы показан на рисунке. Вывод полностью соответствует ожидаемому поведению: условия «жив» и «построено» корректно блокируют действия объектов.

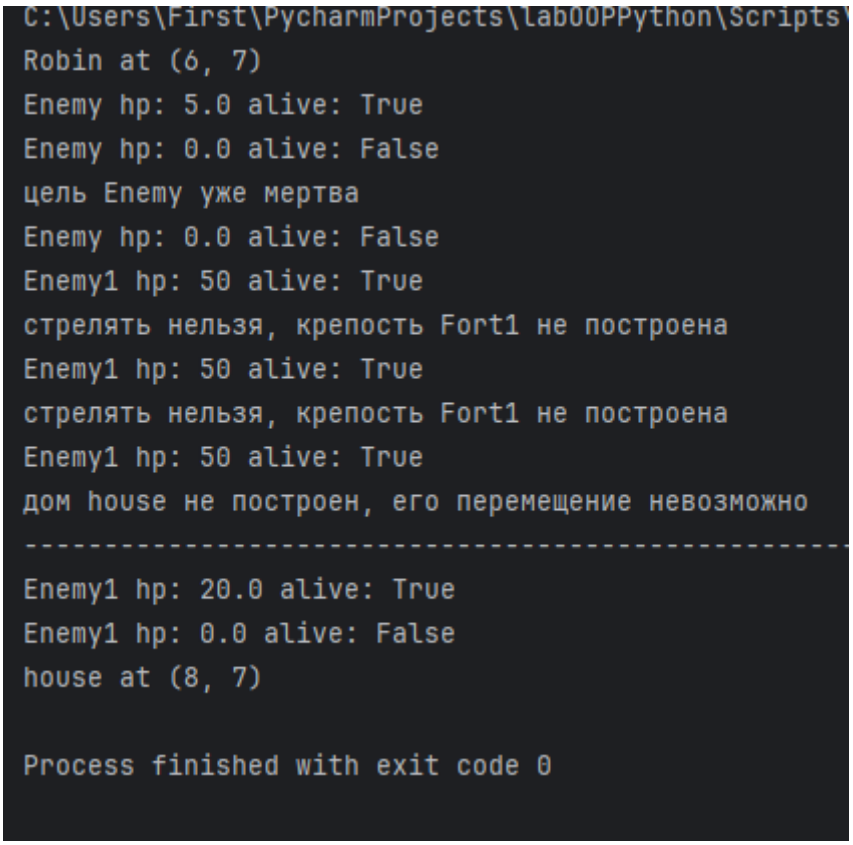



Рис. 1 – Окно консоли с выводом

**Результаты работы**

Проект был опубликован в удалённый репозиторий на GitHub.

**Вывод**

В ходе лабораторной работы я реализовал иерархию классов для пошаговой стратегии на языке Python в среде PyCharm. Все требования задания выполнены. В процессе работы я на практике убедился, что:

1. Абстрактный класс задаёт общий каркас и может содержать состояние (поля, реализацию методов). `GameObject` — именно такой случай: «просто игрового объекта» не бывает, есть конкретные юниты и постройки.
2. Интерфейс описывает роль (контракт) и не содержит состояния. В Python для этого используются абстрактные классы с `@abstractmethod`, а множественное наследование позволяет `Archer` быть одновременно `Unit`, `Attacker` и `Moveable`.
3. Наследование позволило избежать дублирования кода: `Archer` получил идентификатор, имя, координаты и здоровье от родителей `GameObject` и `Unit`.
4. Полиморфизм проявился в методе `Archer.attack(unit)`: он не знает, кто именно перед ним, а просто вызывает `unit.receive_damage(...)`. Если появится юнит с бронёй и переопределит этот метод, код лучника менять не придётся.
5. Инкапсуляция: данные хранятся в защищённых полях (`_hp`, `_x`, `_built` и др.), а доступ идёт через методы. В Python это соглашение, а не жёсткий запрет, но оно показывает, что напрямую менять эти поля не следует.
6. Проверки состояния (`is_alive()`, `is_built()`) не дают объектам попасть в некорректное состояние: мёртвый лучник не ходит и не

атакует, недостроенная крепость не стреляет, недостроенный дом не едет, здоровье не уходит в минус.